E. Security & Privacy Engineering

Standalone capability brief · Suggested module: **Security SDK + Policy Enforcement Platform**

Purpose

Provide reusable secure defaults and enforceable privacy and security controls without allowing shared tooling to replace domain accountability or server-side authorization.

1. Scope and capability areas

Cryptography and key lifecycle

Standardize encryption, hashing, signing, key rotation, and KMS integration. Use approved algorithms and libraries.

Data classification

Label data by sensitivity and define handling rules for storage, transport, access, logging, and retention.

Privacy controls

Support consent, purpose limitation, access/export requests, erasure workflows, and retention enforcement where applicable.

Security policy enforcement

Provide secure headers, SSRF protections, safe parsing, input validation guidance, and secure configuration defaults.

Supply-chain security

Produce SBOMs, sign artifacts, record provenance, scan dependencies, and verify build integrity.

Workload identity

Use short-lived credentials and service identity for service-to-service authentication and authorization.

Abuse prevention

Support rate controls, anomaly signals, bot defenses, suspicious-activity workflows, and tenant-level abuse limits.

2. Shared contracts and standards

- Security-critical decisions are enforced server-side and cannot rely on frontend visibility checks.
- Secrets are never committed to source control, placed in logs, or embedded in client bundles.
- Cryptographic APIs use approved primitives and hide unsafe configuration choices.
- Data classification determines minimum controls for storage, access, telemetry, and retention.
- Privacy requests have traceable ownership, status, and completion evidence.

3. Recommended implementation approach

- Provide standard libraries for identity validation, safe HTTP clients, cryptography, and security headers.
- Integrate secret scanning, dependency scanning, SBOM generation, signing, and provenance verification into CI/CD.
- Use workload identity and short-lived credentials wherever feasible.
- Maintain threat models for platform-critical components and review high-risk changes.

- Define incident response, key compromise, credential rotation, and privacy request runbooks.

4. Failure behavior and operational expectations

- If identity, policy, or key validation cannot be performed safely, fail closed for protected operations.
- Define explicit degraded modes for non-critical controls; never silently bypass critical security enforcement.
- Key rotation failures should alert operators and prevent new use of compromised or expired keys.
- Privacy workflows must report partial completion rather than claiming erasure when downstream copies remain.

5. Suggested operational metrics

- Critical vulnerabilities past SLA
- Percentage of artifacts with verified provenance
- Secret-scanning findings
- Workload identity adoption
- Privacy request completion time
- Security policy bypass incidents

6. Readiness and acceptance checklist

- ■ Protected operations enforce authorization on the server.
- ■ Secrets and cryptographic operations use approved interfaces.
- ■ Artifacts have traceable provenance and software inventory.
- ■ Data handling follows classification and retention policy.
- ■ Security failures have documented fail-closed or controlled-degradation behavior.

Ownership note: assign a named team, service owner, support route, versioning policy, and deprecation process before treating this capability as a platform contract.